

Pet Guardian

Mastering Relational and Non-Relational Database - Sprint 3

Enzo Okuizumi RM 561432

Gustavo Okada RM 563428

Lucas Barros RM 566422

Luna Guimarães RM 562290

Milton Marcelino RM 564836

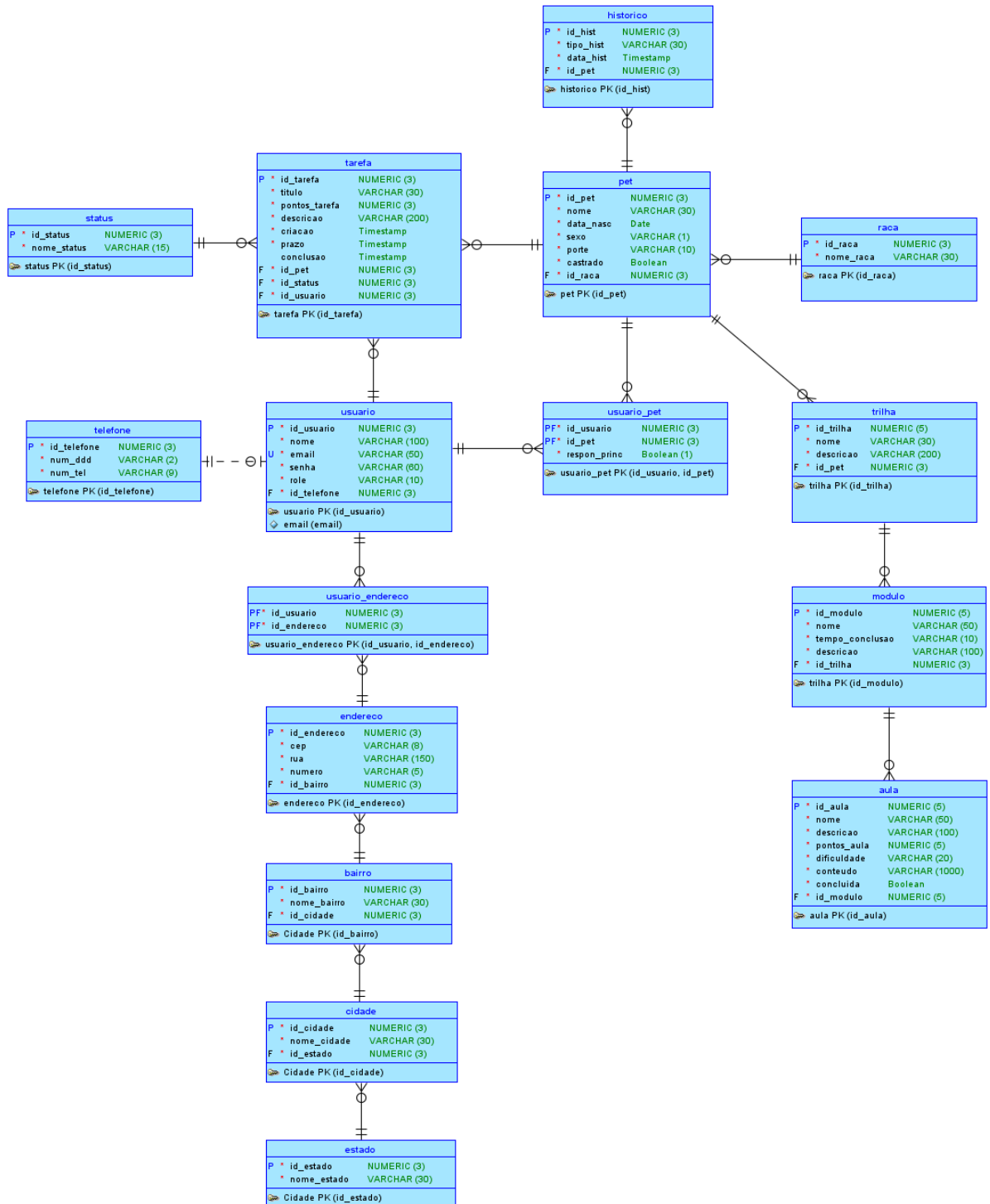
Links Importantes.....	3
Modelo Lógico.....	4
Modelo Relacional.....	5
Tabelas Criadas no Oracle SQL Developer.....	6
DML - Cargas de Dados.....	15
Objetos PL/SQL.....	17
Função 1 — Serialização JSON.....	17
Procedimento 1 — Listagem com JOIN e JSON.....	19
Função 2 — Classificação de Score.....	23
Procedimento 2 — Relatório Analítico Tabular.....	26
Trigger — Auditoria DML.....	29
Bateria de Testes.....	31
Função 1 — fn_tarefa_json.....	31
Procedimento 1 — pr_listar_tarefas_json.....	32
Função 2 — fn_classificar_pontos.....	33
Trigger — trg_audit_tarefa.....	34

Responsáveis pelo desenvolvimento: Lucas Barros / Enzo Okuizumi

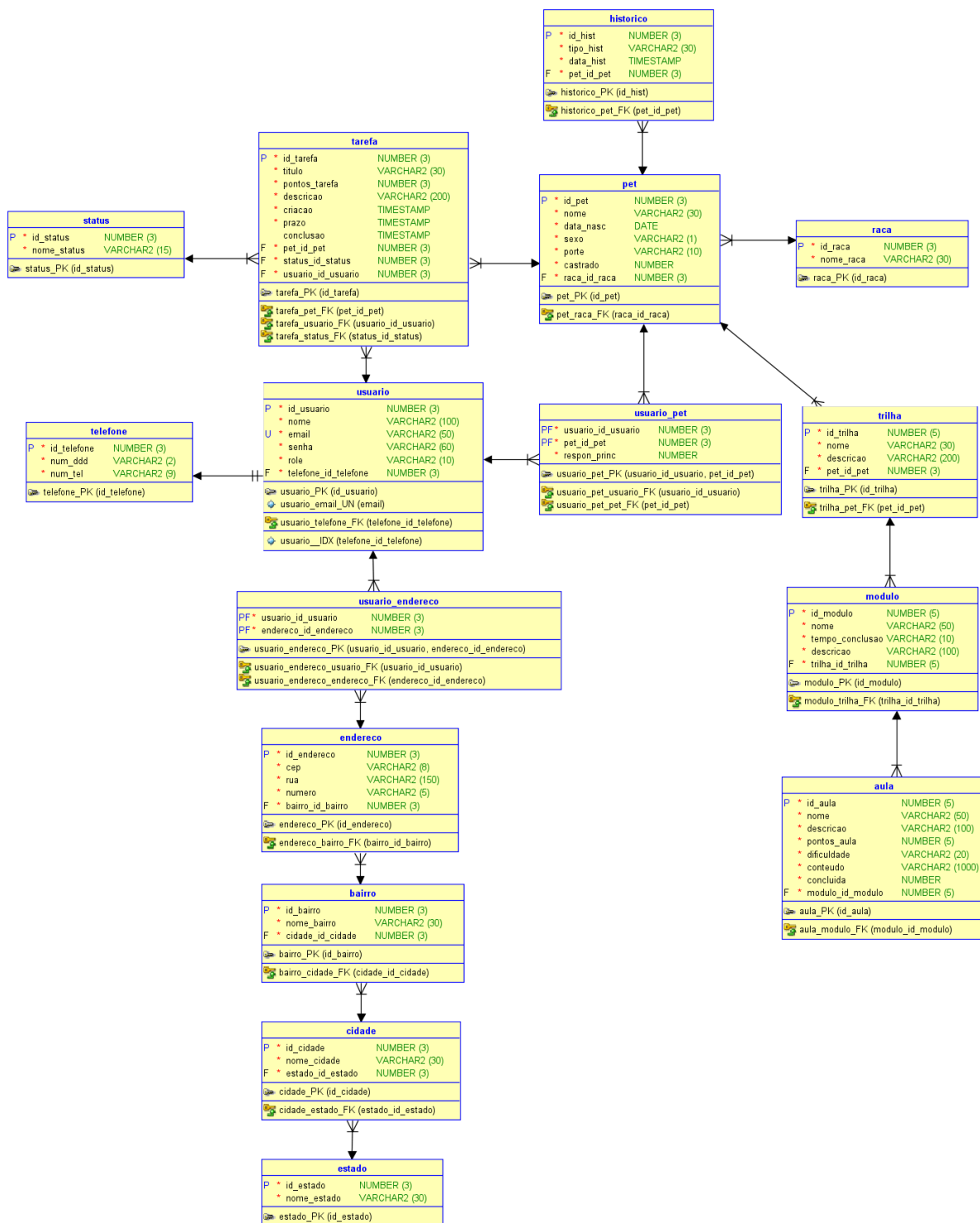
Links Importantes

Repositório Github: [Challenge-Pet-Guardian/Database-Advanced](#)

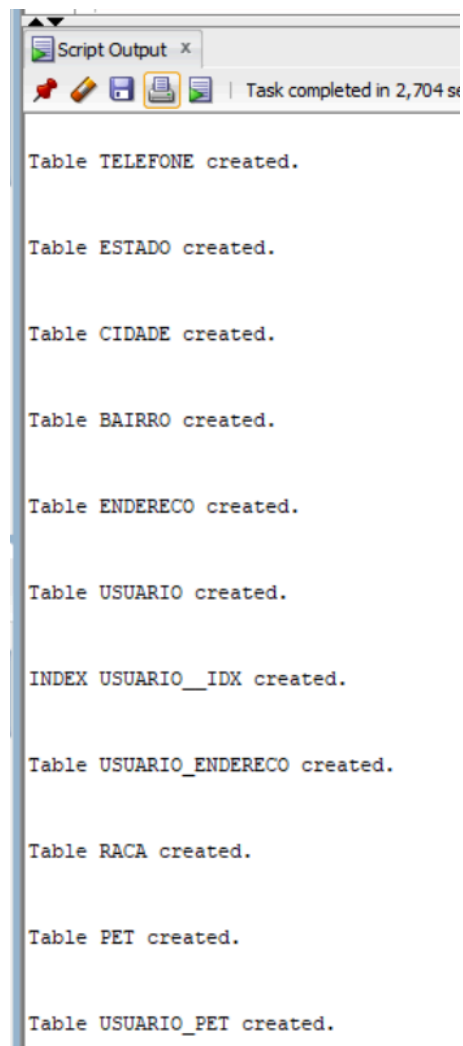
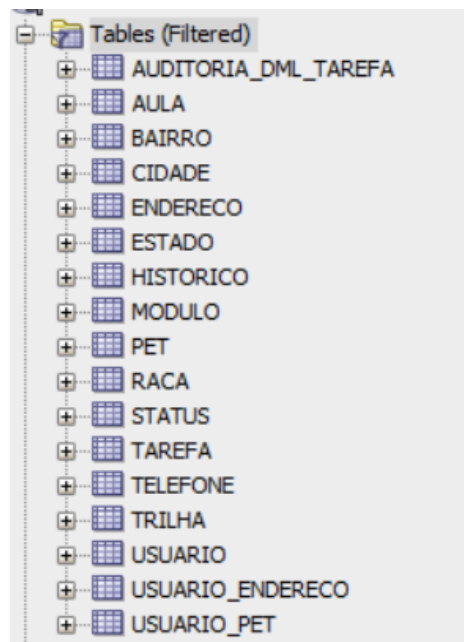
Modelo Lógico

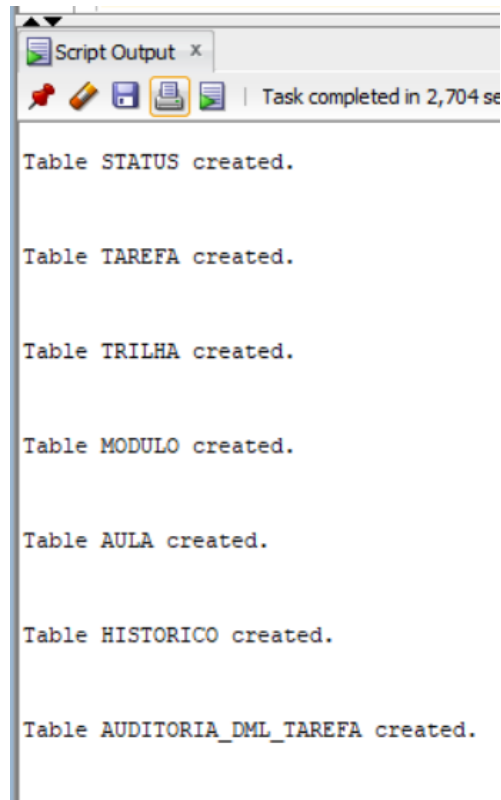


Modelo Relacional



Tabelas Criadas no Oracle SQL Developer





SQL

-- 1. TELEFONE

```
CREATE TABLE telefone (  
    id_telefone NUMBER(3) NOT NULL,  
    num_ddd     VARCHAR2(2) NOT NULL,  
    num_tel     VARCHAR2(9) NOT NULL,  
    CONSTRAINT telefone_pk PRIMARY KEY (id_telefone)  
);
```

SQL

-- 2. ESTADO

```
CREATE TABLE estado (  
    id_estado  NUMBER(3) NOT NULL,  
    nome_estado VARCHAR2(30) NOT NULL,  
    CONSTRAINT estado_pk PRIMARY KEY (id_estado)  
);
```

SQL

-- 3. CIDADE

```
CREATE TABLE cidade (  
    id_cidade          NUMBER(3) NOT NULL,  
    nome_cidade        VARCHAR2(30) NOT NULL,  
    estado_id_estado   NUMBER(3) NOT NULL,  
    CONSTRAINT cidade_pk PRIMARY KEY (id_cidade),  
    CONSTRAINT cidade_estado_fk FOREIGN KEY (estado_id_estado)  
REFERENCES estado(id_estado)  
);
```

SQL

-- 4. BAIRRO

```
CREATE TABLE bairro (  
    id_bairro          NUMBER(3) NOT NULL,  
    nome_bairro        VARCHAR2(30) NOT NULL,  
    cidade_id_cidade   NUMBER(3) NOT NULL,  
    CONSTRAINT bairro_pk PRIMARY KEY (id_bairro),  
    CONSTRAINT bairro_cidade_fk FOREIGN KEY (cidade_id_cidade)  
REFERENCES cidade(id_cidade)  
);
```

SQL

-- 5. ENDEREÇO

```
CREATE TABLE endereco (  
    id_endereco        NUMBER(3) NOT NULL,  
    cep                VARCHAR2(8) NOT NULL,  
    rua                VARCHAR2(150) NOT NULL,  
    numero             VARCHAR2(5) NOT NULL,  
    bairro_id_bairro   NUMBER(3) NOT NULL,  
    CONSTRAINT endereco_pk PRIMARY KEY (id_endereco),  
    CONSTRAINT endereco_bairro_fk FOREIGN KEY (bairro_id_bairro)  
REFERENCES bairro(id_bairro)
```

```
);
```

SQL

-- 6. USUARIO

```
CREATE TABLE usuario (  
    id_usuario          NUMBER(3) NOT NULL,  
    nome                VARCHAR2(100) NOT NULL,  
    email               VARCHAR2(50) NOT NULL,  
    senha               VARCHAR2(60) NOT NULL,  
    role                VARCHAR2(10) NOT NULL,  
    telefone_id_telefone NUMBER(3) NOT NULL,  
    CONSTRAINT usuario_pk PRIMARY KEY (id_usuario),  
    CONSTRAINT usuario_email_un UNIQUE (email),  
    CONSTRAINT ck_usuario_role CHECK (role IN ('ADMIN', 'COMUM',  
'PREMIUM')),  
    CONSTRAINT usuario_telefone_fk FOREIGN KEY  
(telefone_id_telefone) REFERENCES telefone(id_telefone)  
);  
CREATE UNIQUE INDEX usuario__idx ON usuario(telefone_id_telefone  
ASC);
```

SQL

-- 7. USUARIO_ENDEREÇO (N:M)

```
CREATE TABLE usuario_endereco (  
    usuario_id_usuario  NUMBER(3) NOT NULL,  
    endereco_id_endereco NUMBER(3) NOT NULL,  
    CONSTRAINT usuario_endereco_pk PRIMARY KEY  
(usuario_id_usuario, endereco_id_endereco),  
    CONSTRAINT usuario_endereco_usuario_fk FOREIGN KEY  
(usuario_id_usuario) REFERENCES usuario(id_usuario),  
    CONSTRAINT usuario_endereco_endereco_fk FOREIGN KEY  
(endereco_id_endereco) REFERENCES endereco(id_endereco)  
);
```

SQL

-- 8. RACA

```
CREATE TABLE raca (  
    id_raca    NUMBER(3) NOT NULL,  
    nome_raca  VARCHAR2(30) NOT NULL,  
    CONSTRAINT raca_pk PRIMARY KEY (id_raca)  
);
```

SQL

-- 9. PET

```
CREATE TABLE pet (  
    id_pet      NUMBER(3) NOT NULL,  
    nome        VARCHAR2(30) NOT NULL,  
    data_nasc   DATE NOT NULL,  
    sexo        VARCHAR2(1) NOT NULL,  
    porte       VARCHAR2(10) NOT NULL,  
    castrado    NUMBER NOT NULL,  
    raca_id_raca NUMBER(3) NOT NULL,  
    CONSTRAINT pet_pk PRIMARY KEY (id_pet),  
    CONSTRAINT ck_pet_sexo CHECK (sexo IN ('F', 'M')),  
    CONSTRAINT ck_pet_porte CHECK (porte IN ('GRANDE',  
'MEDIO', 'PEQUENO')),  
    CONSTRAINT ck_pet_castrado CHECK (castrado IN (0, 1)),  
    CONSTRAINT pet_raca_fk FOREIGN KEY (raca_id_raca) REFERENCES  
    raca(id_raca)  
);
```

SQL

-- 10. USUARIO_PET (Care Circle N:M)

```
CREATE TABLE usuario_pet (  
    usuario_id_usuario NUMBER(3) NOT NULL,
```

```

    pet_id_pet          NUMBER(3) NOT NULL,
    respon_princ        NUMBER NOT NULL,
    CONSTRAINT usuario_pet_pk    PRIMARY KEY (usuario_id_usuario,
pet_id_pet),
    CONSTRAINT ck_usuario_pet_princ CHECK (respon_princ IN (0,
1)),
    CONSTRAINT usuario_pet_usuario_fk FOREIGN KEY
(usuario_id_usuario) REFERENCES usuario(id_usuario),
    CONSTRAINT usuario_pet_pet_fk    FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet)
);

```

SQL

```

-- 11. STATUS (Domínio Fechado - 3 estados)
CREATE TABLE status (
    id_status    NUMBER(3) NOT NULL,
    nome_status  VARCHAR2(15) NOT NULL,
    CONSTRAINT status_pk PRIMARY KEY (id_status),
    CONSTRAINT ck_status_nome CHECK (nome_status IN ('CONCLUIDO',
'EXPIRADO', 'PENDENTE'))
);

```

SQL

```

-- 12. TAREFA (Tabela de Fatos de Rotina Gamificada)
CREATE TABLE tarefa (
    id_tarefa      NUMBER(3) NOT NULL,
    titulo         VARCHAR2(30) NOT NULL,
    pontos_tarefa  NUMBER(3) NOT NULL,
    descricao      VARCHAR2(200) NOT NULL,
    criacao        TIMESTAMP NOT NULL,
    prazo          TIMESTAMP NOT NULL,
    conclusao      TIMESTAMP,
    pet_id_pet     NUMBER(3) NOT NULL,

```

```

        status_id_status    NUMBER(3) NOT NULL,
        usuario_id_usuario  NUMBER(3) NOT NULL,
        CONSTRAINT tarefa_pk      PRIMARY KEY (id_tarefa),
        CONSTRAINT tarefa_pet_fk   FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet),
        CONSTRAINT tarefa_status_fk FOREIGN KEY (status_id_status)
REFERENCES status(id_status),
        CONSTRAINT tarefa_usuario_fk FOREIGN KEY (usuario_id_usuario)
REFERENCES usuario(id_usuario)
);

```

SQL

-- 13. TRILHA

```

CREATE TABLE trilha (
    id_trilha    NUMBER(5) NOT NULL,
    nome         VARCHAR2(30) NOT NULL,
    descricao    VARCHAR2(200) NOT NULL,
    pet_id_pet   NUMBER(3) NOT NULL,
    CONSTRAINT trilha_pk      PRIMARY KEY (id_trilha),
    CONSTRAINT trilha_pet_fk  FOREIGN KEY (pet_id_pet) REFERENCES
pet(id_pet)
);

```

SQL

-- 14. MODULO

```

CREATE TABLE modulo (
    id_modulo     NUMBER(5) NOT NULL,
    nome          VARCHAR2(50) NOT NULL,
    tempo_conclusao VARCHAR2(10) NOT NULL,
    descricao     VARCHAR2(100) NOT NULL,
    trilha_id_trilha NUMBER(5) NOT NULL,
    CONSTRAINT modulo_pk      PRIMARY KEY (id_modulo),

```

```
        CONSTRAINT modulo_trilha_fk FOREIGN KEY (trilha_id_trilha)
REFERENCES trilha(id_trilha)
);
```

SQL

-- 15. AULA

```
CREATE TABLE aula (
    id_aula          NUMBER(5) NOT NULL,
    nome             VARCHAR2(50) NOT NULL,
    descricao        VARCHAR2(100) NOT NULL,
    pontos_aula      NUMBER(5) NOT NULL,
    dificuldade      VARCHAR2(20) NOT NULL,
    conteudo         VARCHAR2(1000) NOT NULL,
    concluida        NUMBER NOT NULL,
    modulo_id_modulo NUMBER(5) NOT NULL,
    CONSTRAINT aula_pk          PRIMARY KEY (id_aula),
    CONSTRAINT ck_aula_concluida CHECK (concluida IN (0, 1)),
    CONSTRAINT aula_modulo_fk   FOREIGN KEY (modulo_id_modulo)
REFERENCES modulo(id_modulo)
);
```

SQL

-- 16. HISTORICO

```
CREATE TABLE historico (
    id_hist    NUMBER(3) NOT NULL,
    tipo_hist  VARCHAR2(30) NOT NULL,
    data_hist  TIMESTAMP NOT NULL,
    pet_id_pet NUMBER(3) NOT NULL,
    CONSTRAINT historico_pk          PRIMARY KEY (id_hist),
    CONSTRAINT historico_pet_fk     FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet)
);
```

SQL

```
-- TABELA DE AUDITORIA DML
CREATE TABLE auditoria_dml_tarefa (
    id_auditoria      NUMBER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
    nome_usuario      VARCHAR2(50) NOT NULL,
    tipo_operacao      VARCHAR2(10) NOT NULL,
    data_hora_operacao TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    valoresanteriores VARCHAR2(4000),
    valoresnovos       VARCHAR2(4000)
);
```

DML - Cargas de Dados

```
Enzo
Worksheet Query Builder

-- 1. TELEFONE (5+ registros)
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (1, '11', '981234567');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (2, '11', '972345678');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (3, '11', '963456789');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (4, '11', '954567890');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (5, '11', '945678901');

-- 2. ESTADO (5+ registros)
INSERT INTO estado (id_estado, nome_estado) VALUES (1, 'São Paulo');
INSERT INTO estado (id_estado, nome_estado) VALUES (2, 'Rio de Janeiro');
INSERT INTO estado (id_estado, nome_estado) VALUES (3, 'Minas Gerais');
INSERT INTO estado (id_estado, nome_estado) VALUES (4, 'Paraná');
INSERT INTO estado (id_estado, nome_estado) VALUES (5, 'Santa Catarina');

-- 3. CIDADE (5+ registros)
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (1, 'São Paulo', 1);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (2, 'Campinas', 1);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (3, 'Rio de Janeiro', 2);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (4, 'Belo Horizonte', 3);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (5, 'Curitiba', 4);

-- 4. BAIRRO (5+ registros)
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (1, 'Bela Vista', 1);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (2, 'Pinheiros', 1);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (3, 'Cambui', 2);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (4, 'Copacabana', 3);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (5, 'Savassi', 4);

-- 5. ENDEREÇO (5+ registros)
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (1, '01311000', 'Avenida Paulista', '1000', 1);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (2, '05422000', 'Rua dos Pinheiros', '250', 2);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (3, '13024000', 'Rua Coronel Silva Telles', '85', 3);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (4, '22070010', 'Avenida Atlântica', '1500', 4);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES (5, '30140010', 'Avenida Getúlio Vargas', '300', 5);
```

```
Enzo
Worksheet Query Builder

-- 6. USUARIO (5+ registros com senhas legíveis)
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (1, 'Carlos Silva', 'carlos.silva@email.com', 'Senha#123', 'PREMIUM', 1);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (2, 'Mariana Oliveira', 'mariana.o@email.com', 'Senha#456', 'COMUM', 2);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (3, 'Roberto Santos', 'roberto.s@email.com', 'Senha#789', 'ADMIN', 3);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (4, 'Fernanda Costa', 'fernanda.c@email.com', 'Senha#101', 'COMUM', 4);
INSERT INTO usuario (id_usuario, nome, email, senha, role, telefone_id_telefone)
VALUES (5, 'Lucas Mendes', 'lucas.m@email.com', 'Senha#202', 'PREMIUM', 5);

-- 7. USUARIO_ENDEREÇO (5+ registros)
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (1, 1);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (2, 2);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (3, 3);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (4, 4);
INSERT INTO usuario_endereco (usuario_id_usuario, endereco_id_endereco) VALUES (5, 5);

-- 8. RACA (5+ registros)
INSERT INTO raca (id_raca, nome_raca) VALUES (1, 'Vira-lata (SRD)');
INSERT INTO raca (id_raca, nome_raca) VALUES (2, 'Golden Retriever');
INSERT INTO raca (id_raca, nome_raca) VALUES (3, 'Pug');
INSERT INTO raca (id_raca, nome_raca) VALUES (4, 'Gato Persa');
INSERT INTO raca (id_raca, nome_raca) VALUES (5, 'Gato Siamês');

-- 9. PET (5+ registros)
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (1, 'Thor', DATE '2021-04-10', 'M', 'GRANDE', 1, 2);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (2, 'Mel', DATE '2019-08-15', 'F', 'MEDIO', 1, 1);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (3, 'Bob', DATE '2022-01-20', 'M', 'PEQUENO', 0, 3);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (4, 'Mimi', DATE '2020-11-05', 'F', 'PEQUENO', 1, 4);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
```

```
10. USUARIO_PET (Care Circle familiar - 5+ registros)
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (1, 1, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (1, 2, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (2, 3, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (3, 4, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (4, 5, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES (5, 1, 0);

11. STATUS (Justificativa Técnica de Domínio Fechado / Finite State Machine)
-- NOTA ARQUITETURAL: A tabela STATUS atua como uma Máquina de Estados Finita e canônica para o ciclo
-- de vida de cuidados do pet. Ela é estritamente restrita aos 3 estados discretos do sistema, validada
-- pela constraint 'ck_status_nome CHECK (nome_status IN ('CONCLUIDO', 'EXPIRADO', 'PENDENTE'))'.
-- Inserir estados adicionais violaria o domínio fechado e a integridade de negócio da aplicação.
-- Todas as demais tabelas relacionais do projeto contêm 5 ou mais registros válidos.

INSERT INTO status (id_status, nome_status) VALUES (1, 'PENDENTE');
INSERT INTO status (id_status, nome_status) VALUES (2, 'CONCLUIDO');
INSERT INTO status (id_status, nome_status) VALUES (3, 'EXPIRADO');

12. TAREFA (8 registros com variação de Pet e Status)
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (1, 'Passeio Matinal', 25, 'Caminhada de 40 minutos no parque', SYSTIMESTAMP - INTERVAL '2' DAY, SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP - INTERVAL '1' DAY, NULL, 1, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (2, 'Medicamento Antibiótico', 50, 'Administrar 1 comprimido pós-refeição', SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 1, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (3, 'Escovação de Pelos', 15, 'Remoção de subpelos mortos com rasqueteira', SYSTIMESTAMP - INTERVAL '3' DAY, SYSTIMESTAMP - INTERVAL '2' DAY, NULL, 1, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (4, 'Alimentação Especial', 20, 'Ração sênior com suplementação vitamínica', SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 2, 1);
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (5, 'Banho e Higiene Bucal', 60, 'Banho morno com xampu neutro e escovação dental', SYSTIMESTAMP - INTERVAL '4' DAY, SYSTIMESTAMP - INTERVAL '3' DAY, NULL, 1, 1);
```

```
13. TRILHA (5+ registros)
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (1, 'Boas Maneiras Caninas', 'Comandos básicos de obediência e disciplina', 1);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (2, 'Socialização Filhotes', 'Introdução segura a outros animais e ambientes', 2);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (3, 'Controle de Ansiedade', 'Redução do estresse de separação em cães de guarda', 3);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (4, 'Enriquecimento Felino', 'Gatificação e estímulos sensoriais verticais', 4);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (5, 'Adaptação de Ambientes', 'Criação de rotinas calmas e seguras para gatos', 5);

14. MODULO (5+ registros)
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (1, 'Comando Senta e Fica', '45 min', 'Técnicas de reforço positivo cc', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (2, 'Passeio Sem Puxar a Guia', '60 min', 'Equipamentos adequados e cc', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (3, 'Apresentação a Novos Cães', '30 min', 'Leitura da linguagem corpo', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (4, 'Brinquedos Recheáveis', '40 min', 'Uso de kong e tapetes de lamb', trilha_id_trilha);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (5, 'Instalação de Nichos', '50 min', 'Posicionamento estratégico de p', trilha_id_trilha);

15. AULA (5+ registros)
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (1, 'Introdução ao Clicker', 'Fundamentos de marcação positiva', 50, 'INICIANTE', 'Aprenda a associar o clique ao reforço primário.', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (2, 'Critério de Duração do Fica', 'Aumentando o tempo gradativamente', 80, 'INTERMEDIARIO', 'Suba os segundos antes de entregar a recompensa.', 0, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (3, 'Troca de Foco na Guia', 'Atenção ao tutor em vez de estímulos da rua', 70, 'INTERMEDIARIO', 'Exercício do U-Turn quando houver tensão na guia.', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (4, 'Receitas de Enriquecimento', 'Preparo de petiscos congelados saudáveis', 40, 'INICIANTE', 'Receita de purê de abóbora e caldo de carne congelado.', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, conteudo, concluida, modulo_id_modulo)
VALUES (5, 'Rotas de Fuga e Segurança', 'Evitando que o felino se sinta acuado', 60, 'AVANÇADO', 'Planejamento de rotas circulares em cômodos integrados.', 0, 1);

16. HISTORICO (5+ registros)
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (1, 'Vacina Polivalente V10', SYSTIMESTAMP - INTERVAL '90' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (2, 'Check-up Clínico Preventivo', SYSTIMESTAMP - INTERVAL '45' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (3, 'Antipulgas HexGard', SYSTIMESTAMP - INTERVAL '20' DAY, 2);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (4, 'Limpeza de Tártaro', SYSTIMESTAMP - INTERVAL '60' DAY, 3);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (5, 'Vacina Ovídenimela Felina V6', SYSTIMESTAMP - INTERVAL '120' DAY, 4);
```

Output Exibindo Inserções

```
Script Output x
Task completed in 2,704 seconds

1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```

Objetos PL/SQL

Função 1 — Serialização JSON

Nome: fn_tarefa_json

Assinatura: fn_tarefa_json(p_id_tarefa IN NUMBER, p_titulo IN VARCHAR2, p_pontos IN NUMBER, p_descricao IN VARCHAR2, p_nome_pet IN VARCHAR2, p_nome_status IN VARCHAR2, p_nome_usuario IN VARCHAR2) RETURN VARCHAR2

Recebe dados relacionais de uma tarefa extraídos via JOIN e monta o documento JSON caractere a caractere — sem uso de nenhuma função built-in do Oracle para JSON (TO_JSON, JSON_OBJECT, etc.), garantindo alto desacoplamento e eliminando queries redundantes.

Exceções tratadas:

Exceção	Código ORA	Condição
e_id_nulo_invalido	-20001	ID nulo ou ≤ 0
e_dados_incompletos	-20002	Atributos relacionais obrigatórios nulos
e_json_excedente	-20003	JSON gerado excede 3800 bytes
OTHERS	-20004	Qualquer erro não previsto

SQL

```
CREATE OR REPLACE FUNCTION fn_tarefa_json (  
    p_id_tarefa      IN NUMBER,  
    p_titulo         IN VARCHAR2,  
    p_pontos         IN NUMBER,  
    p_descricao      IN VARCHAR2,  
    p_nome_pet       IN VARCHAR2,  
    p_nome_status    IN VARCHAR2,  
    p_nome_usuario   IN VARCHAR2  
) RETURN VARCHAR2 IS  
    v_json           VARCHAR2(4000);
```

-- Declaração de exceções personalizadas de regra de negócio

```

e_id_invalido      EXCEPTION;
e_dados_incompletos EXCEPTION;
e_json_excedente   EXCEPTION;
BEGIN
    -- Validação de ID
    IF p_id_tarefa IS NULL OR p_id_tarefa <= 0 THEN
        RAISE e_id_invalido;
    END IF;

    -- Validação de consistência dos dados relacionais obrigatórios
    IF p_titulo IS NULL OR p_nome_pet IS NULL OR p_nome_status IS
    NULL OR p_nome_usuario IS NULL THEN
        RAISE e_dados_incompletos;
    END IF;

    -- Construção manual e determinística do documento JSON (sem
    qualquer função built-in)
    v_json := '{' ||
        '"id_tarefa":' || p_id_tarefa || ',' ||
        '"titulo":"' || REPLACE(REPLACE(p_titulo, '\',
'\'), '"', '\"') || ',' ||
        '"pontos":' || NVL(p_pontos, 0) || ',' ||
        '"descricao":"' || REPLACE(REPLACE(NVL(p_descricao,
''), '\', '\\'), '"', '\"') || ',' ||
        '"pet":"' || REPLACE(REPLACE(p_nome_pet, '\',
'\'), '"', '\"') || ',' ||
        '"status":"' || p_nome_status || ',' ||
        '"cuidador_responsavel":"' ||
REPLACE(REPLACE(p_nome_usuario, '\', '\\'), '"', '\"') || ' ' ||
        '}' ;

    -- Validação de estouro de buffer seguro de saída
    IF LENGTH(v_json) > 3800 THEN
        RAISE e_json_excedente;
    END IF;

    RETURN v_json;

```

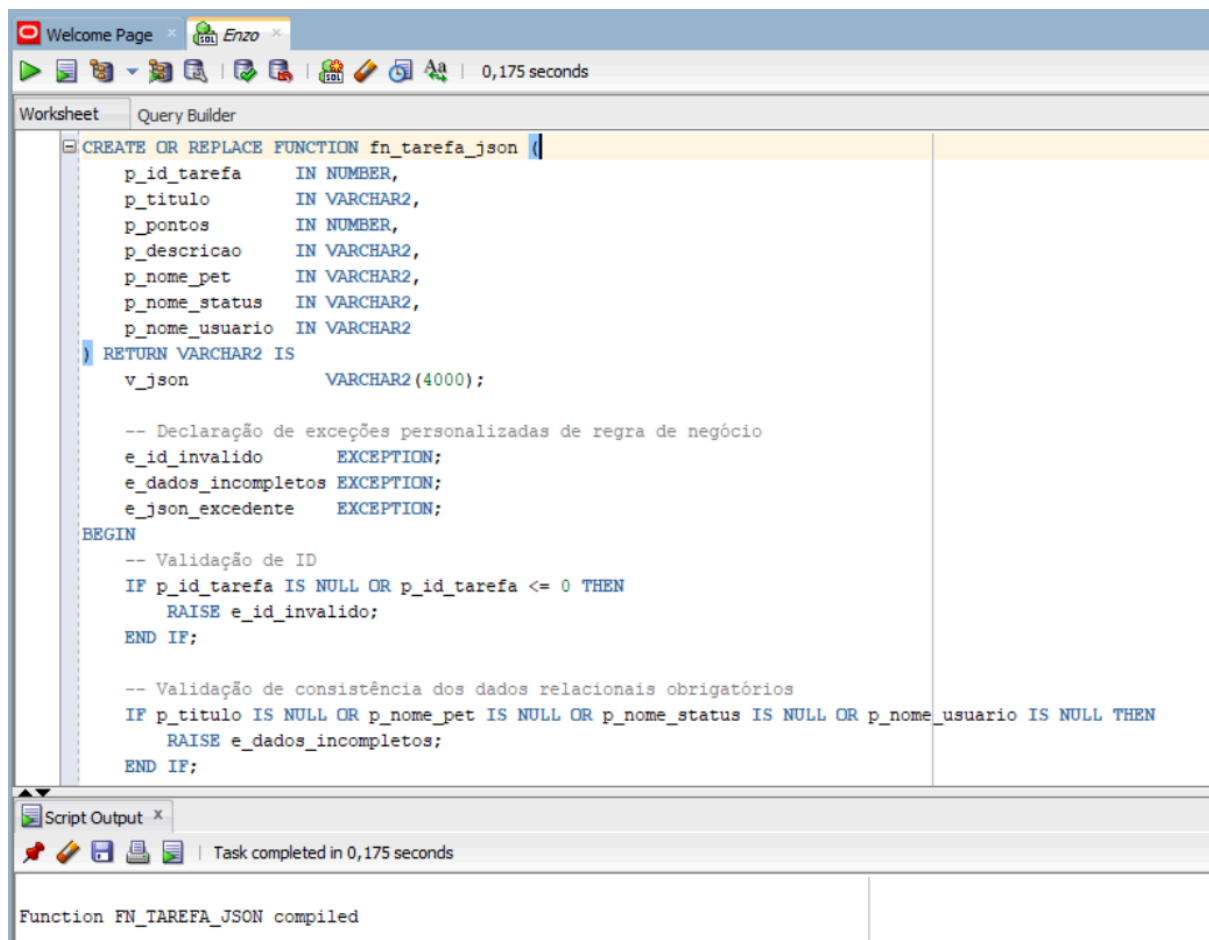
EXCEPTION

```
    WHEN e_id_invalido THEN
        RAISE_APPLICATION_ERROR(-20001, 'ID da tarefa fornecido é
nulo ou inválido (deve ser > 0).');
    WHEN e_dados_incompletos THEN
        RAISE_APPLICATION_ERROR(-20002, 'Atributos relacionais
obrigatórios da tarefa não podem ser nulos.');
```

```
    WHEN e_json_excedente THEN
        RAISE_APPLICATION_ERROR(-20003, 'Estrutura JSON manual
excedeu o limite máximo seguro de 3800 bytes.');
```

```
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20004, 'Falha inesperada durante
a serialização manual JSON: ' || SQLERRM);
END fn_tarefa_json;
/
```

Prints



The screenshot shows the Oracle SQL Developer interface. The top pane displays a PL/SQL procedure named `pr_listar_tarefas_json`. The procedure has two main sections: a success case and an exception case. The success case calls `fn_tarefa_json` with parameters for task ID, title, points, description, pet name, status, and user. The exception case handles an invalid task ID by outputting an error message. The bottom pane shows the script output, indicating that the procedure was completed successfully and that an exception was captured for an invalid task ID.

```

-- Caso de Sucesso:
DECLARE
v_resultado VARCHAR2(4000);
BEGIN
v_resultado := fn_tarefa_json(
p_id_tarefa => 1,
p_titulo => 'Passeio Matinal',
p_pontos => 25,
p_descricao => 'Caminhada de 40 minutos no parque',
p_nome_pet => 'Thor',
p_nome_status => 'CONCLUIDO',
p_nome_usuario => 'Carlos Silva'
);
DBMS_OUTPUT.PUT_LINE('JSON Gerado: ' || v_resultado);
END;
/

-- Caso de Exceção Tratada (ID Inválido):
BEGIN
DBMS_OUTPUT.PUT_LINE(fn_tarefa_json(-1, 'Teste', 10, 'Desc', 'Pet', 'STATUS', 'User'));
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Exceção capturada com sucesso: ' || SQLERRM);
END;
/

```

Script Output:

```

Task completed in 0,098 seconds

JSON Gerado: {"id_tarefa":1,"titulo":"Passeio Matinal","pontos":25,"descricao":"Caminhada de 40 minutos no parque","pet":"Thor","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}

PL/SQL procedure successfully completed.

Exceção capturada com sucesso: ORA-20001: ID da tarefa fornecido é nulo ou inválido (deve ser > 0).

PL/SQL procedure successfully completed.

```

Procedimento 1 — Listagem com JOIN e JSON

Nome: `pr_listar_tarefas_json`

Assinatura: `pr_listar_tarefas_json(p_status_id IN NUMBER DEFAULT NULL)`

Abre cursor com JOIN explícito entre 4 tabelas relacionais (tarefa, pet, status, usuario), itera sobre cada linha e delega a serialização passando a tupla relacional para a `fn_tarefa_json` sem requisições redundantes (N+1 queries).

Exceções tratadas:

Exceção	Código ORA	Condição
e_status_inexistente	-20005	ID de status não existe na tabela
e_sem_registros	-20006	Nenhuma tarefa retornada
CURSOR_ALREADY_OPEN	-20007	Cursor já aberto em outra sessão
OTHERS	-20008	Qualquer erro não previsto

SQL

```

CREATE OR REPLACE PROCEDURE pr_listar_tarefas_json (
    p_status_id IN NUMBER DEFAULT NULL
) IS

```

```

-- Cursor com JOIN explícito entre Tarefa, Pet, Status e
Usuario
CURSOR c_tarefas_join IS
    SELECT t.id_tarefa,
           t.titulo,
           t.pontos_tarefa,
           t.descricao,
           p.nome AS nome_pet,
           s.nome_status,
           u.nome AS nome_usuario
    FROM tarefa t
    JOIN pet p      ON t.pet_id_pet = p.id_pet
    JOIN status s   ON t.status_id_status = s.id_status
    JOIN usuario u  ON t.usuario_id_usuario = u.id_usuario
    WHERE p_status_id IS NULL OR t.status_id_status =
p_status_id
    ORDER BY t.id_tarefa;

r_tarefa          c_tarefas_join%ROWTYPE;
v_json_gerado     VARCHAR2(4000);
v_total_linhas    NUMBER := 0;

-- Exceções personalizadas
e_status_inexistente EXCEPTION;
e_sem_registros    EXCEPTION;
v_check_status     NUMBER;
BEGIN
    -- Validação: se status for informado, checar se ele existe
na base
    IF p_status_id IS NOT NULL THEN
        SELECT COUNT(1) INTO v_check_status FROM status WHERE
id_status = p_status_id;
        IF v_check_status = 0 THEN
            RAISE e_status_inexistente;
        END IF;
    END IF;
END IF;

```

```

DBMS_OUTPUT.PUT_LINE('=====
=====');
        DBMS_OUTPUT.PUT_LINE('          RELATÓRIO DE TAREFAS
SERIALIZADAS EM JSON (MANUAL)          ');

DBMS_OUTPUT.PUT_LINE('=====
=====');

OPEN c_tarefas_join;
LOOP
    FETCH c_tarefas_join INTO r_tarefa;
    EXIT WHEN c_tarefas_join%NOTFOUND;

    -- Delega serialização à Função 1 passando os dados
relacionais extraídos pelo JOIN
    v_json_gerado := fn_tarefa_json(
        r_tarefa.id_tarefa,
        r_tarefa.titulo,
        r_tarefa.pontos_tarefa,
        r_tarefa.descricao,
        r_tarefa.nome_pet,
        r_tarefa.nome_status,
        r_tarefa.nome_usuario
    );
    DBMS_OUTPUT.PUT_LINE(v_json_gerado);
    v_total_linhas := v_total_linhas + 1;
END LOOP;
CLOSE c_tarefas_join;

IF v_total_linhas = 0 THEN
    RAISE e_sem_registros;
END IF;

DBMS_OUTPUT.PUT_LINE('-----
-----');
        DBMS_OUTPUT.PUT_LINE('Total de registros JSON exibidos com
sucesso: ' || v_total_linhas);

```

```

DBMS_OUTPUT.PUT_LINE('=====
=====');

EXCEPTION
    WHEN e_status_inexistente THEN
        RAISE_APPLICATION_ERROR(-20005, 'O ID de status informado
(' || p_status_id || ') não existe na tabela STATUS.');
```

```

    WHEN e_sem_registros THEN
        RAISE_APPLICATION_ERROR(-20006, 'Nenhuma tarefa atende
aos critérios de filtro informados.');
```

```

    WHEN CURSOR_ALREADY_OPEN THEN
        RAISE_APPLICATION_ERROR(-20007, 'O cursor de consulta de
tarefas já se encontra aberto em outra sessão.');
```

```

    WHEN OTHERS THEN
        IF c_tarefas_join%ISOPEN THEN
            CLOSE c_tarefas_join;
        END IF;
        RAISE_APPLICATION_ERROR(-20008, 'Erro inesperado na
execução do Procedimento 1: ' || SQLERRM);
END pr_listar_tarefas_json;
/
```

Prints

The screenshot displays the SQL Developer environment. The top toolbar includes icons for running scripts, saving, and other standard database operations. The 'Worksheet' tab is active, showing a PL/SQL procedure named 'pr_listar_tarefas_json'. The procedure is designed to accept an optional parameter 'p_status_id' and uses an explicit JOIN cursor to retrieve task details along with related pet, status, and user information. The procedure includes error handling for missing status IDs, no matching records, and cursor already open scenarios. The bottom pane shows the 'Script Output' window, indicating that the procedure was compiled successfully and the task was completed in 0.058 seconds.

```

CREATE OR REPLACE PROCEDURE pr_listar_tarefas_json (
    p_status_id IN NUMBER DEFAULT NULL
) IS
    -- Cursor com JOIN explícito entre Tarefa, Pet, Status e Usuario
    CURSOR c_tarefas_join IS
        SELECT t.id_tarefa,
               t.titulo,
               t.pontos_tarefa,
               t.descricao,
               p.nome AS nome_pet,
               s.nome_status,
               u.nome AS nome_usuario
        FROM tarefa t
        JOIN pet p      ON t.pet_id_pet = p.id_pet
        JOIN status s    ON t.status_id_status = s.id_status
        JOIN usuario u   ON t.usuario_id_usuario = u.id_usuario
        WHERE p_status_id IS NULL OR t.status_id_status = p_status_id
        ORDER BY t.id_tarefa;

    r_tarefa      c_tarefas_join%ROWTYPE;
    v_json_gerado VARCHAR2(4000);
    v_total_linhas NUMBER := 0;

```

Task completed in 0,058 seconds

Procedure PR_LISTAR_TAREFAS_JSON compiled

```

-- Caso de Sucesso (Listar Todos):
BEGIN
  pr_listar_tarefas_json(NULL);
END;
/

-- Caso de Exceção Tratada (Status Inexistente):
BEGIN
  pr_listar_tarefas_json(99);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Exceção capturada com sucesso: ' || SQLERRM);
END;

```

Task completed in 0,101 seconds

RELATÓRIO DE TAREFAS SERIALIZADAS EM JSON (MANUAL)

```

[{"id_tarefa":1,"titulo":"Passeio Matinal","pontoss":25,"descricao":"Caminhada de 40 minutos no parque","pet":"Thor","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":2,"titulo":"Medicamento Antibiótico","pontoss":50,"descricao":"Administrar 1 comprimido pós-refeição","pet":"Thor","status":"PENDENTE","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":3,"titulo":"Escovação de Pelos","pontoss":15,"descricao":"Remoção de subpelos mortos com rasqueteadeira","pet":"Thor","status":"EXPIRADO","cuidador_responsavel":"Lucas Mendes"}
{"id_tarefa":4,"titulo":"Alimentação Especial","pontoss":20,"descricao":"Ração sênior com suplementação vitamínica","pet":"Mel","status":"PENDENTE","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":5,"titulo":"Banho e Higiene Bucal","pontoss":60,"descricao":"Banho morno com xampu neutro e escovação dental","pet":"Mel","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":6,"titulo":"Corte de Unhas","pontoss":30,"descricao":"Aparar as pontas das unhas com alicate apropriado","pet":"Bob","status":"PENDENTE","cuidador_responsavel":"Mariana Oliveira"}
{"id_tarefa":7,"titulo":"Limpeza da Caixa de Areia","pontoss":20,"descricao":"Higienização completa e troca da sílica","pet":"Mimi","status":"CONCLUIDO","cuidador_responsavel":"Roberto Santos"}
{"id_tarefa":8,"titulo":"Estímulo com Varinha","pontoss":15,"descricao":"Sessão de brincadeiras de caça por 20 minutos","pet":"Luna","status":"PENDENTE","cuidador_responsavel":"Fernanda Costa"}
]

```

Total de registros JSON exibidos com sucesso: 8

PL/SQL procedure successfully completed.

Exceção capturada com sucesso: ORA-20005: O ID de status informado (99) não existe na tabela STATUS.

Função 2 — Classificação de Score

Nome: fn_classificar_pontos

Assinatura: fn_classificar_pontos(p_pontos IN NUMBER) RETURN VARCHAR2

Implementa a regra de negócio de gamificação de bem-estar do PetGuardian, retornando a faixa de classificação do pet conforme a pontuação.

Regras de classificação:

Faixa de Pontos	Classificação
0 – 20	BRONZE (BÁSICO)
21 – 45	PRATA (INTERMEDIÁRIO)
46 – 75	OURO (AVANÇADO)
76 – 1000	DIAMANTE (MASTER)

Exceções tratadas:

Exceção	Código ORA	Condição
e_pontos_nulo	-20009	Pontuação nula
e_pontos_negativo	-20010	Pontuação < 0
e_pontos_excesso	-20011	Pontuação > 1000
OTHERS	-20012	Qualquer erro não previsto

SQL

```
CREATE OR REPLACE FUNCTION fn_classificar_pontos ( p_pontos IN
NUMBER) RETURN VARCHAR2 IS
    v_classificacao VARCHAR2(50);
    e_pontos_nulo EXCEPTION;
    e_pontos_negativo EXCEPTION;
    e_pontos_excesso EXCEPTION;
BEGIN
    IF p_pontos IS NULL THEN
        RAISE e_pontos_nulo;
    ELSIF p_pontos < 0 THEN
        RAISE e_pontos_negativo;
    ELSIF p_pontos > 1000 THEN
        RAISE e_pontos_excesso;
    END IF;

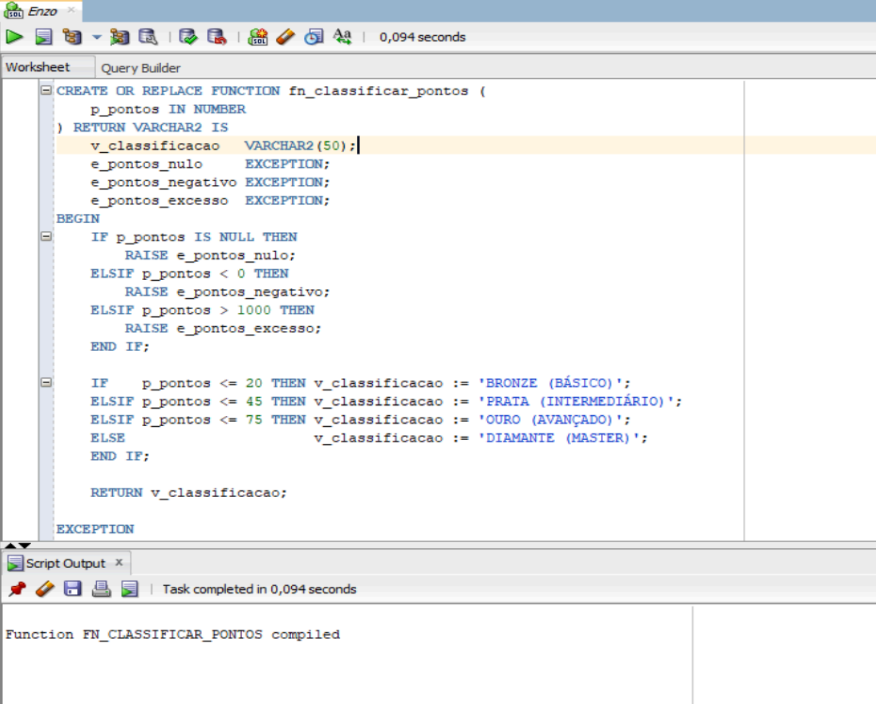
    IF p_pontos <= 20 THEN
        v_classificacao := 'BRONZE (BÁSICO)';
    ELSIF p_pontos <= 45 THEN
        v_classificacao := 'PRATA (INTERMEDIÁRIO)';
    ELSIF p_pontos <= 75 THEN
        v_classificacao := 'OURO (AVANÇADO)';
    ELSE
        v_classificacao := 'DIAMANTE (MASTER)';
    END IF;

    RETURN v_classificacao;
EXCEPTION
    WHEN e_pontos_nulo THEN
        RAISE_APPLICATION_ERROR(-20009, 'Pontuação informada não
pode ser nula.');
```

```
    WHEN e_pontos_negativo THEN
        RAISE_APPLICATION_ERROR(-20010, 'A pontuação não pode ser
negativa: ' || p_pontos);
    WHEN e_pontos_excesso THEN
        RAISE_APPLICATION_ERROR(-20011, 'Pontuação acima do
limite máximo permitido de 1000 pontos: ' || p_pontos);
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20012, 'Erro ao classificar
pontos na Função 2: ' || SQLERRM);
```

```
END fn_classificar_pontos;  
/
```

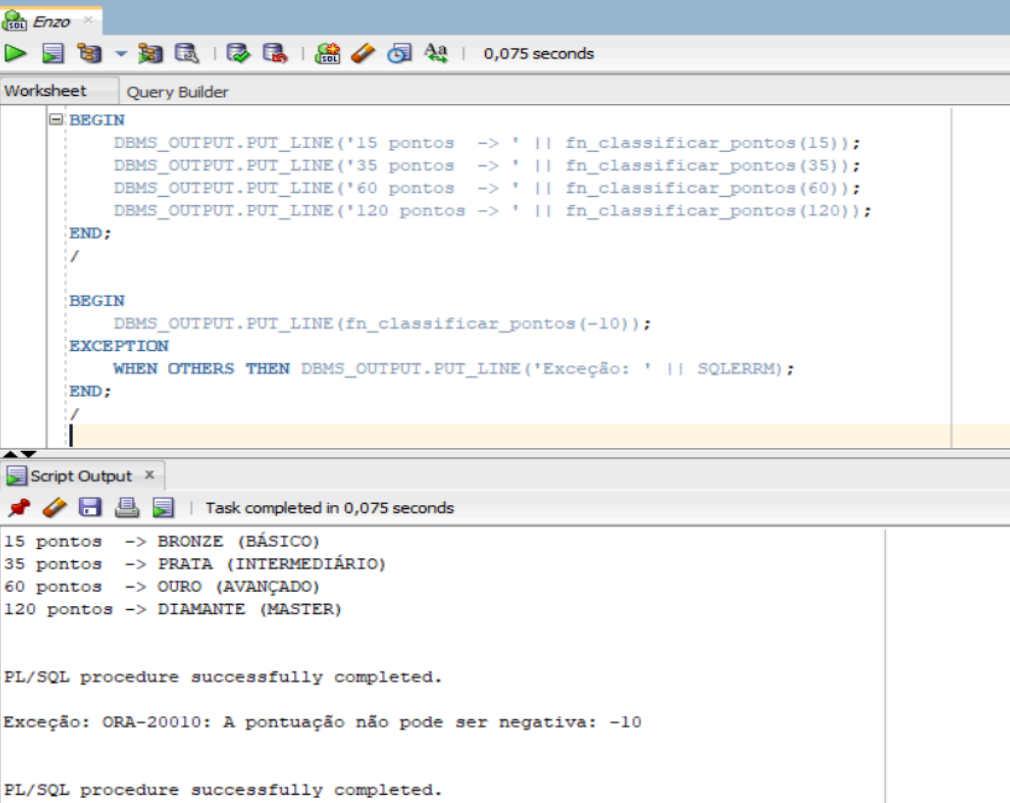
Prints



The screenshot shows the Enzo IDE interface. The top toolbar includes icons for running, saving, and other standard IDE functions. The 'Worksheet' tab is active, displaying the SQL code for creating the function. The 'Query Builder' tab is also visible. The code defines a function that takes a number of points and returns a classification level, with exceptions for null, negative, and excessive values.

```
CREATE OR REPLACE FUNCTION fn_classificar_pontos (  
    p_pontos IN NUMBER  
) RETURN VARCHAR2 IS  
    v_classificacao VARCHAR2(50);  
    e_pontos_nulo EXCEPTION;  
    e_pontos_negativo EXCEPTION;  
    e_pontos_excesso EXCEPTION;  
  
BEGIN  
    IF p_pontos IS NULL THEN  
        RAISE e_pontos_nulo;  
    ELSIF p_pontos < 0 THEN  
        RAISE e_pontos_negativo;  
    ELSIF p_pontos > 1000 THEN  
        RAISE e_pontos_excesso;  
    END IF;  
  
    IF p_pontos <= 20 THEN v_classificacao := 'BRONZE (BÁSICO)';  
    ELSIF p_pontos <= 45 THEN v_classificacao := 'PRATA (INTERMEDIÁRIO)';  
    ELSIF p_pontos <= 75 THEN v_classificacao := 'OURO (AVANÇADO)';  
    ELSE  
        v_classificacao := 'DIAMANTE (MASTER)';  
    END IF;  
  
    RETURN v_classificacao;  
  
EXCEPTION
```

Script Output x
Task completed in 0,094 seconds
Function FN_CLASSIFICAR_PONTOS compiled



The screenshot shows the Enzo IDE interface. The top toolbar includes icons for running, saving, and other standard IDE functions. The 'Worksheet' tab is active, displaying the SQL code for testing the function. The 'Query Builder' tab is also visible. The code uses DBMS_OUTPUT.PUT_LINE to print the results of the function for various input values. The 'Script Output' tab shows the results of the execution, including the classification levels for different point values and an error message for a negative input.

```
BEGIN  
    DBMS_OUTPUT.PUT_LINE('15 pontos -> ' || fn_classificar_pontos(15));  
    DBMS_OUTPUT.PUT_LINE('35 pontos -> ' || fn_classificar_pontos(35));  
    DBMS_OUTPUT.PUT_LINE('60 pontos -> ' || fn_classificar_pontos(60));  
    DBMS_OUTPUT.PUT_LINE('120 pontos -> ' || fn_classificar_pontos(120));  
END;  
/  
  
BEGIN  
    DBMS_OUTPUT.PUT_LINE(fn_classificar_pontos(-10));  
EXCEPTION  
    WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Exceção: ' || SQLERRM);  
END;  
/
```

Script Output x
Task completed in 0,075 seconds
15 pontos -> BRONZE (BÁSICO)
35 pontos -> PRATA (INTERMEDIÁRIO)
60 pontos -> OURO (AVANÇADO)
120 pontos -> DIAMANTE (MASTER)

PL/SQL procedure successfully completed.

Exceção: ORA-20010: A pontuação não pode ser negativa: -10

PL/SQL procedure successfully completed.

Procedimento 2 — Relatório Analítico Tabular

Nome: pr_resumo_pontos_tarefas

Parâmetros: nenhum

Gera relatório tabular de pontos de tarefas com dois níveis de agrupamento (Control Break): Pet (Cat 1) e Status (Cat 2). O somatório por combinação, subtotal por Pet e total geral são calculados via variáveis acumuladoras em PL/SQL. A formatação de colunas usa RPAD/LPAD.

Exceções tratadas:

Exceção	Código ORA	Condição
e_sem_fatos_cadastrados	-20013	Cursor retornou zero registros
VALUE_ERROR	-20014	Estouro numérico na conversão
CURSOR_ALREADY_OPEN	-20017	Cursor analítico já se encontra aberto
OTHERS	-20015	Erro fatal genérico

SQL

```
CREATE OR REPLACE PROCEDURE pr_resumo_pontos_tarefas IS
  -- Cursor de fatos detalhados sem agregação no motor SQL
  CURSOR c_fatos IS
    SELECT p.id_pet,
           p.nome AS nome_pet,
           s.id_status,
           s.nome_status,
           t.pontos_tarefa
    FROM tarefa t
    JOIN pet p    ON t.pet_id_pet = p.id_pet
    JOIN status s ON t.status_id_status = s.id_status
    ORDER BY p.id_pet ASC, s.id_status ASC;

  r_linha c_fatos%ROWTYPE;

  -- Variáveis de controle de quebra de grupo em 2 níveis
  (Control Break)
  v_pet_atual      pet.id_pet%TYPE := NULL;
  v_nome_pet_atual pet.nome%TYPE  := NULL;
  v_status_atual   status.id_status%TYPE := NULL;
  v_nome_status_atual status.nome_status%TYPE := NULL;
```

```

-- Acumuladores em PL/SQL
v_soma_combinacao      NUMBER(10,2) := 0;
v_subtotal_pet         NUMBER(10,2) := 0;
v_total_geral          NUMBER(10,2) := 0;
v_qtd_linhas           NUMBER := 0;

-- Exceções distintas
e_sem_fatos_cadastrados EXCEPTION;
BEGIN
    OPEN c_fatos;

    -- Cabeçalho com colunas: Pet (Cat 1), Status (Cat 2) e
    Pontos
    DBMS_OUTPUT.PUT_LINE(RPAD('Pet (Cat 1)', 18) || ' ' ||
RPAD('Status (Cat 2)', 16) || ' ' || LPAD('Pontos', 12));
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 18, '-') || ' ' ||
RPAD('-', 16, '-') || ' ' || LPAD('-', 12, '-'));

    LOOP
        FETCH c_fatos INTO r_linha;

        -- Quebra de combinação (Pet ou Status) ou término do
        cursor
        IF (c_fatos%NOTFOUND OR r_linha.id_pet <> v_pet_atual OR
r_linha.id_status <> v_status_atual)
            AND v_pet_atual IS NOT NULL THEN

            -- Exibe linha consolidada da combinação (Pet,
            Status, Soma)
            DBMS_OUTPUT.PUT_LINE(
                RPAD(TO_CHAR(v_pet_atual) || ' - ' ||
v_nome_pet_atual, 18) || ' ' ||
                RPAD(TO_CHAR(v_status_atual) || ' - ' ||
v_nome_status_atual, 16) || ' ' ||
                LPAD(TO_CHAR(v_soma_combinacao, 'FM999990.00'),
12)
                );

```

```

        -- Propaga acumuladores para subtotal e total geral
        v_subtotal_pet      := v_subtotal_pet +
v_soma_combinacao;
        v_total_geral      := v_total_geral +
v_soma_combinacao;
        v_soma_combinacao := 0;

        -- Quebra de Pet: emite Sub Total
        IF c_fatos%NOTFOUND OR r_linha.id_pet <> v_pet_atual
THEN
            DBMS_OUTPUT.PUT_LINE(RPAD('Sub Total', 35) ||
LPAD(TO_CHAR(v_subtotal_pet, 'FM999990.00'), 12));
            v_subtotal_pet := 0;
        END IF;
    END IF;

    EXIT WHEN c_fatos%NOTFOUND;

    -- Armazena o estado atual dos agrupamentos
    v_pet_atual      := r_linha.id_pet;
    v_nome_pet_atual := r_linha.nome_pet;
    v_status_atual   := r_linha.id_status;
    v_nome_status_atual := r_linha.nome_status;

    -- Acumula pontos da linha atual
    v_soma_combinacao := v_soma_combinacao +
r_linha.pontos_tarefa;
    v_qtd_linhas      := v_qtd_linhas + 1;
END LOOP;

CLOSE c_fatos;

-- Validação de ausência total de registros
IF v_qtd_linhas = 0 THEN
    RAISE e_sem_fatos_cadastrados;
END IF;

-- 3. Emite o Total Geral final na última linha da tabela

```

```

        DBMS_OUTPUT.PUT_LINE(RPAD('Total Geral', 35) ||
LPAD(TO_CHAR(v_total_geral, 'FM999990.00'), 12));

EXCEPTION
    WHEN e_sem_fatos_cadastrados THEN
        RAISE_APPLICATION_ERROR(-20013, 'Nenhum registro de
tarefa/fato localizado para sumarização.');
```

WHEN VALUE_ERROR THEN

```

        RAISE_APPLICATION_ERROR(-20014, 'Estouro de capacidade ou
erro de conversão numérica durante o somatório manual.');
```

WHEN CURSOR_ALREADY_OPEN THEN

```

        RAISE_APPLICATION_ERROR(-20017, 'O cursor analítico de
fatos já se encontra aberto nesta sessão.');
```

WHEN OTHERS THEN

```

        IF c_fatos%ISOPEN THEN
            CLOSE c_fatos;
        END IF;
        RAISE_APPLICATION_ERROR(-20015, 'Erro fatal no cálculo do
Procedimento 2: ' || SQLERRM);
END pr_resumo_pontos_tarefas;
/
```

Prints

The screenshot displays the SQL Developer environment. The main window is the Query Builder, showing the SQL code for the procedure `pr_resumo_pontos_tarefas`. The code defines a cursor `c_fatos` that selects data from the `tarefa` table, joined with `pet` and `status` tables. The script output window at the bottom indicates that the task was completed in 0,065 seconds and the procedure `PR_RESUMO_PONTOS_TAREFAS` was successfully compiled.

```

CREATE OR REPLACE PROCEDURE pr_resumo_pontos_tarefas IS
-- Cursor de fatos detalhados sem agregação no motor SQL
CURSOR c_fatos IS
    SELECT p.id_pet,
           p.nome AS nome_pet,
           s.id_status,
           s.nome_status,
           t.pontos_tarefa
    FROM tarefa t
    JOIN pet p ON t.pet_id_pet = p.id_pet
    JOIN status s ON t.status_id_status = s.id_status
    ORDER BY p.id_pet ASC, s.id_status ASC;

    r_linha c_fatos%ROWTYPE;

-- Variáveis de controle de quebra de grupo em 2 níveis (Control Break)

```

Script Output x

Task completed in 0,065 seconds

Procedure PR_RESUMO_PONTOS_TAREFAS compiled

Welcome Page Enzo

Worksheet Query Builder

```
-- 1. Caso de Sucesso: Execução do Relatório Tabular
BEGIN
  pr_resumo_pontos_tarefas;
END;
/

-- 2. Caso de Exceção Tratada: Indução de e_sem_fatos_cadastrados sem alterar o banco
BEGIN
  SAVEPOINT sp_teste_proc2;
  -- Limpa temporariamente a tabela de fatos na sessão
  DELETE FROM tarefa;
  pr_resumo_pontos_tarefas;
  ROLLBACK TO sp_teste_proc2;
```

Script Output x

Task completed in 0,201seconds

Pet (Cat 1)	Status (Cat 2)	Pontos
1 - Thor	1 - PENDENTE	50.00
1 - Thor	2 - CONCLUIDO	25.00
1 - Thor	3 - EXPIRADO	15.00
Sub Total		90.00
2 - Mel	1 - PENDENTE	20.00
2 - Mel	2 - CONCLUIDO	60.00
Sub Total		80.00
3 - Bob	1 - PENDENTE	30.00
Sub Total		30.00
4 - Mimi	2 - CONCLUIDO	20.00
Sub Total		20.00
5 - Luna	1 - PENDENTE	15.00
Sub Total		15.00
Total Geral		235.00

PL/SQL procedure successfully completed.

Pet (Cat 1)	Status (Cat 2)	Pontos
Exceção capturada com sucesso: ORA-20013: Nenhum registro de tarefa/fato localizado para sumarização.		

Trigger — Auditoria DML

Nome: trg_audit_tarefa

Evento: AFTER INSERT OR UPDATE OR DELETE ON tarefa FOR EACH ROW

Tabela de destino: auditoria_dml_tarefa

Captura automaticamente cada operação DML na tabela TAREFA, registrando: usuário de banco (USER), tipo de operação, timestamp e os valores anteriores (:OLD) e novos (:NEW) dos campos relevantes.

SQL

```
CREATE OR REPLACE TRIGGER trg_audit_tarefa
AFTER INSERT OR UPDATE OR DELETE ON tarefa
FOR EACH ROW
DECLARE
  v_tipo_operacao VARCHAR2(10);
```

```

v_valores_old    VARCHAR2(4000) := NULL;
v_valores_new    VARCHAR2(4000) := NULL;
BEGIN
    IF INSERTING THEN
        v_tipo_operacao := 'INSERT';
        v_valores_new := 'ID_TAREFA=' || :NEW.id_tarefa || ';
TITULO=' || :NEW.titulo || '; PONTOS=' || :NEW.pontos_tarefa ||
'; STATUS=' || :NEW.status_id_status || '; PET=' ||
:NEW.pet_id_pet || '; USUARIO=' || :NEW.usuario_id_usuario;
        INSERT INTO auditoria_dml_tarefa (nome_usuario,
tipo_operacao, data_hora_operacao, valores_anteriores,
valores_novos)
            VALUES (USER, v_tipo_operacao, SYSTIMESTAMP, NULL,
v_valores_new);
    ELSIF UPDATING THEN
        v_tipo_operacao := 'UPDATE';
        v_valores_old := 'ID_TAREFA=' || :OLD.id_tarefa || ';
TITULO=' || :OLD.titulo || '; PONTOS=' || :OLD.pontos_tarefa ||
'; STATUS=' || :OLD.status_id_status || '; PET=' ||
:OLD.pet_id_pet || '; USUARIO=' || :OLD.usuario_id_usuario;
        v_valores_new := 'ID_TAREFA=' || :NEW.id_tarefa || ';
TITULO=' || :NEW.titulo || '; PONTOS=' || :NEW.pontos_tarefa ||
'; STATUS=' || :NEW.status_id_status || '; PET=' ||
:NEW.pet_id_pet || '; USUARIO=' || :NEW.usuario_id_usuario;
        INSERT INTO auditoria_dml_tarefa (nome_usuario,
tipo_operacao, data_hora_operacao, valores_anteriores,
valores_novos)
            VALUES (USER, v_tipo_operacao, SYSTIMESTAMP,
v_valores_old, v_valores_new);
    ELSIF DELETING THEN
        v_tipo_operacao := 'DELETE';
        v_valores_old := 'ID_TAREFA=' || :OLD.id_tarefa || ';
TITULO=' || :OLD.titulo || '; PONTOS=' || :OLD.pontos_tarefa ||
'; STATUS=' || :OLD.status_id_status || '; PET=' ||
:OLD.pet_id_pet || '; USUARIO=' || :OLD.usuario_id_usuario;
        INSERT INTO auditoria_dml_tarefa (nome_usuario,
tipo_operacao, data_hora_operacao, valores_anteriores,
valores_novos)

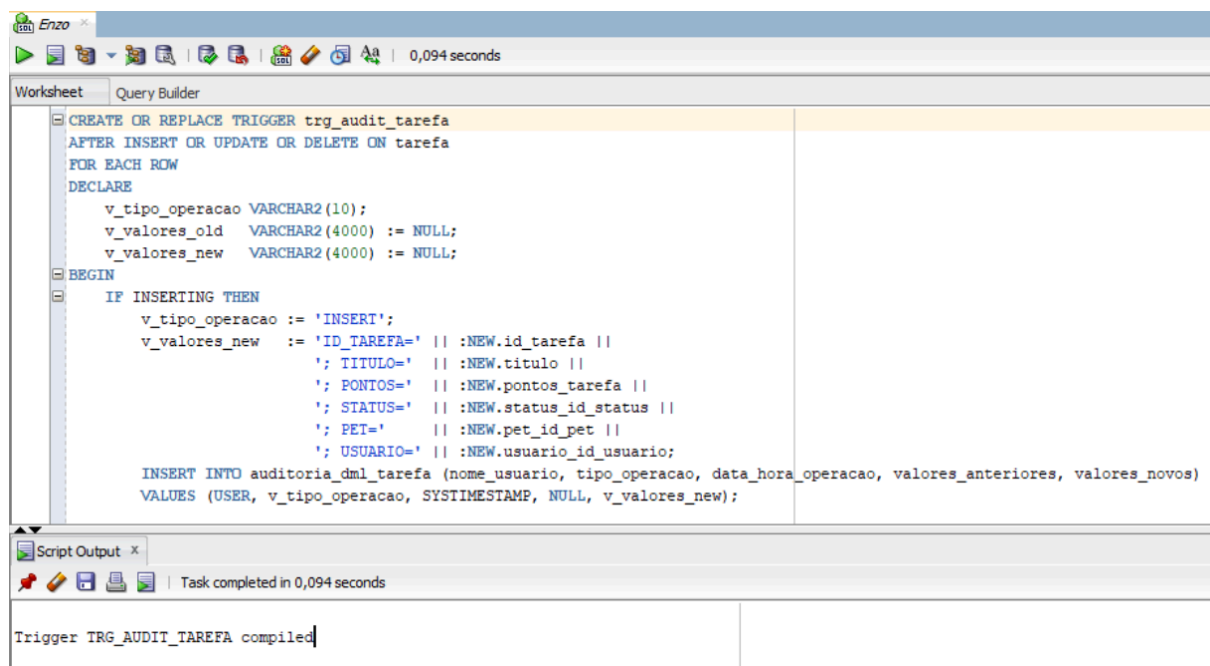
```

```

VALUES (USER, v_tipo_operacao, SYSTIMESTAMP,
v_valores_old, NULL);
END IF;
EXCEPTION
WHEN OTHERS THEN
RAISE_APPLICATION_ERROR(-20016, 'Erro fatal na execução
da trigger: ' || SQLERRM);
END trg_audit_tarefa;
/

```

Prints



Bateria de Testes

Função 1 — fn_tarefa_json

Caso	Entrada	Resultado Esperado
Sucesso	p_id_tarefa = 1	JSON com dados da tarefa ID 1
Exceção — ID inexistente	p_id_tarefa = 999	ORA-20002

Prints

```

-- 4.1 TESTE DA FUNÇÃO 1 (Serialização JSON Manual a partir de Dados Relacionais)
PROMPT === TESTE FUNÇÃO 1: Caso de Sucesso ===;
DECLARE
    v_resultado VARCHAR2(4000);
BEGIN
    v_resultado := fn_tarefa_json(
        p_id_tarefa => 1,
        p_titulo => 'Passeio Matinal',
        p_pontos => 25,
        p_descricao => 'Caminhada de 40 minutos no parque',
        p_nome_pet => 'Thor',
        p_nome_status => 'CONCLUIDO',
        p_nome_usuario => 'Carlos Silva'
    );
    DBMS_OUTPUT.PUT_LINE('JSON Gerado: ' || v_resultado);
END;
/

PROMPT === TESTE FUNÇÃO 1: Caso de Exceção Tratada (ID Inválido) ===;
BEGIN
    DBMS_OUTPUT.PUT_LINE(fn_tarefa_json(-1, 'Teste', 10, 'Desc', 'Pet', 'STATUS', 'User'));
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Exceção capturada com sucesso: ' || SQLERRM);
END;
/

==== TESTE FUNÇÃO 1: Caso de Sucesso ====
JSON Gerado: {"id_tarefa":1,"titulo":"Passeio Matinal","pontos":25,"descricao":"Caminhada de 40 minutos no parque","pet":"Thor","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}

PL/SQL procedure successfully completed.

==== TESTE FUNÇÃO 1: Caso de Exceção Tratada (ID Inválido) ====
Exceção capturada com sucesso: ORA-20001: ID da tarefa fornecido é nulo ou inválido (deve ser > 0).
```

Procedimento 1 — pr_listar_tarefas_json

Caso	Entrada	Resultado Esperado
Sucesso	p_status_id = NULL	Todos os JSONs + total de registros
Exceção — status inexistente	p_status_id = 99	ORA-20005

Prints

```

PROMPT === TESTE PROCEDIMENTO 1: Caso de Sucesso (Todos os registros) ===;
BEGIN
    pr_listar_tarefas_json(NULL);
END;
/

PROMPT === TESTE PROCEDIMENTO 1: Caso de Exceção Tratada (Status Inexistente) ===;
BEGIN
    pr_listar_tarefas_json(99);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Exceção capturada com sucesso: ' || SQLERRM);
END;
/

==== TESTE PROCEDIMENTO 1: Caso de Sucesso (Todos os registros) ====
RELATÓRIO DE TAREFAS SERIALIZADAS EM JSON (MANUAL)
=====
{"id_tarefa":1,"titulo":"Passeio Matinal","pontos":25,"descricao":"Caminhada de 40 minutos no parque","pet":"Thor","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":2,"titulo":"Medicamento Antibiótico","pontos":50,"descricao":"Administrar 1 comprimido pós-refeição","pet":"Thor","status":"PENDENTE","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":3,"titulo":"Escovação de Pelos","pontos":15,"descricao":"Remoção de subpelos mortos com raqueteira","pet":"Thor","status":"EXPIRADO","cuidador_responsavel":"Lucas Mendes"}
{"id_tarefa":4,"titulo":"Alimentação Especial","pontos":20,"descricao":"Ração sênior com suplementação vitamínica","pet":"Mel","status":"PENDENTE","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":5,"titulo":"Banho e Higiene Bucal","pontos":60,"descricao":"Banho morno com xampu neutro e escovação dental","pet":"Mel","status":"CONCLUIDO","cuidador_responsavel":"Carlos Silva"}
{"id_tarefa":6,"titulo":"Corte de Unhas","pontos":30,"descricao":"Aparar as pontas das unhas com alicate apropriado","pet":"Bob","status":"PENDENTE","cuidador_responsavel":"Mariana Oliveira"}
{"id_tarefa":7,"titulo":"Limpeza da Caixa de Areia","pontos":20,"descricao":"Higienização completa e troca da sílica","pet":"Mimi","status":"CONCLUIDO","cuidador_responsavel":"Roberto Santos"}
{"id_tarefa":8,"titulo":"Estímulo com Varinha","pontos":15,"descricao":"Sessão de brincadeiras de caça por 20 minutos","pet":"Luna","status":"PENDENTE","cuidador_responsavel":"Fernanda Costa"}
=====
Total de registros JSON exibidos com sucesso: 8

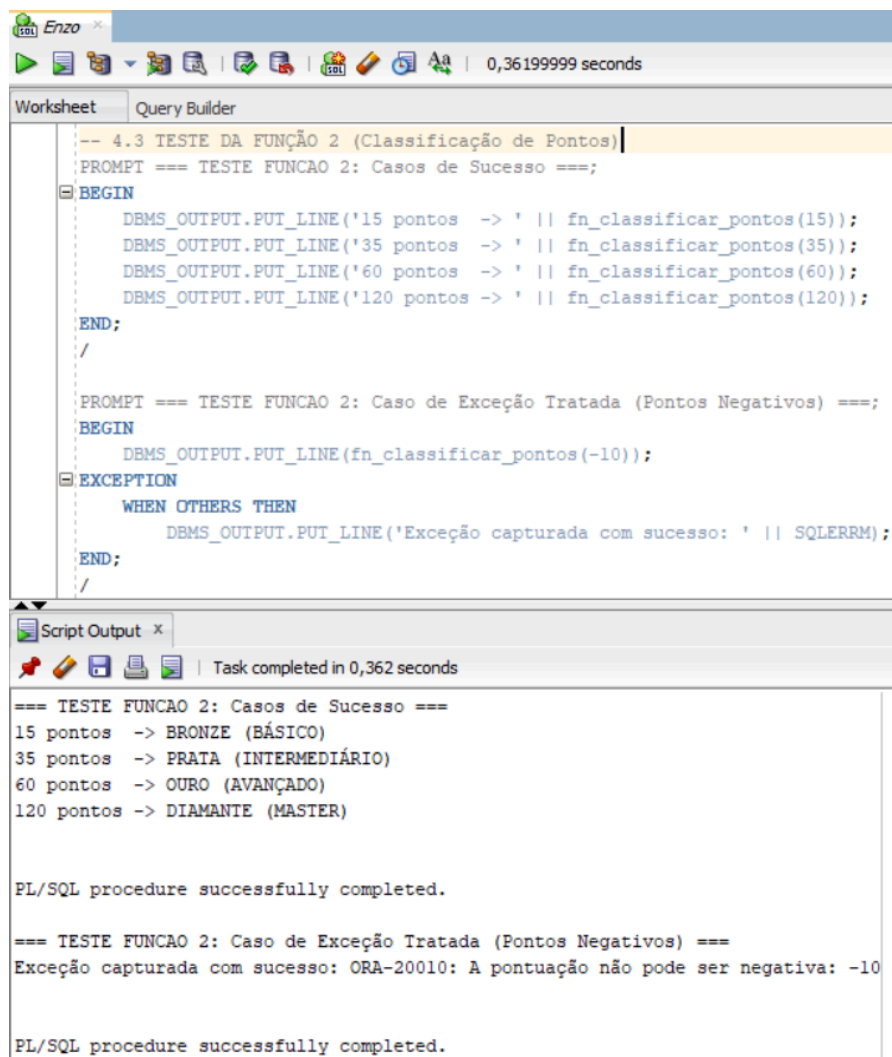
PL/SQL procedure successfully completed.

==== TESTE PROCEDIMENTO 1: Caso de Exceção Tratada (Status Inexistente) ====
Exceção capturada com sucesso: ORA-20005: O ID de status informado (99) não existe na tabela STATUS.
```

Função 2 — fn_classificar_pontos

Caso	Entrada	Resultado Esperado
Sucesso — BRONZE	15	BRONZE (BÁSICO)
Sucesso — PRATA	35	PRATA (INTERMEDIÁRIO)
Sucesso — OURO	60	OURO (AVANÇADO)
Sucesso — DIAMANTE	120	DIAMANTE (MASTER)
Exceção — negativo	-10	ORA-20010

Prints



The screenshot displays the Enzo IDE interface. The top toolbar includes icons for running, saving, and other standard IDE functions, along with a timer showing 0,36199999 seconds. The main window is titled 'Worksheet' and 'Query Builder'. It contains a PL/SQL script for testing the 'fn_classificar_pontos' function. The script is divided into two sections: one for successful cases (BRONZE, PRATA, OURO, DIAMANTE) and another for an exception case (negative points). The output window, titled 'Script Output', shows the results of the script execution, including the expected classification for each point value and a message indicating that the PL/SQL procedure was successfully completed. The exception case shows the error message 'ORA-20010: A pontuação não pode ser negativa: -10'.

```
-- 4.3 TESTE DA FUNÇÃO 2 (Classificação de Pontos)
PROMPT === TESTE FUNCAO 2: Casos de Sucesso ===;
BEGIN
    DBMS_OUTPUT.PUT_LINE('15 pontos -> ' || fn_classificar_pontos(15));
    DBMS_OUTPUT.PUT_LINE('35 pontos -> ' || fn_classificar_pontos(35));
    DBMS_OUTPUT.PUT_LINE('60 pontos -> ' || fn_classificar_pontos(60));
    DBMS_OUTPUT.PUT_LINE('120 pontos -> ' || fn_classificar_pontos(120));
END;
/

PROMPT === TESTE FUNCAO 2: Caso de Exceção Tratada (Pontos Negativos) ===;
BEGIN
    DBMS_OUTPUT.PUT_LINE(fn_classificar_pontos(-10));
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Exceção capturada com sucesso: ' || SQLERRM);
END;
/
```

```
=== TESTE FUNCAO 2: Casos de Sucesso ===
15 pontos -> BRONZE (BÁSICO)
35 pontos -> PRATA (INTERMEDIÁRIO)
60 pontos -> OURO (AVANÇADO)
120 pontos -> DIAMANTE (MASTER)

PL/SQL procedure successfully completed.

=== TESTE FUNCAO 2: Caso de Exceção Tratada (Pontos Negativos) ===
Exceção capturada com sucesso: ORA-20010: A pontuação não pode ser negativa: -10

PL/SQL procedure successfully completed.
```

Procedimento 2 — pr_resumo_pontos_tarefas

Pet (Cat 1)	Status (Cat 2)	Pontos
1 - Thor	1 - PENDENTE	50.00
1 - Thor	2 - CONCLUIDO	25.00
1 - Thor	3 - EXPIRADO	15.00
Sub Total		90.00
2 - Mel	1 - PENDENTE	20.00
2 - Mel	2 - CONCLUIDO	60.00
Sub Total		80.00
3 - Bob	1 - PENDENTE	30.00
Sub Total		30.00
4 - Mimi	2 - CONCLUIDO	20.00
Sub Total		20.00
5 - Luna	1 - PENDENTE	15.00
Sub Total		15.00
Total Geral		235.00

Worksheet Query Builder

```

PROMPT === TESTE PROCEDIMENTO 2: Execução com Subtotais Manuais e Total Geral ===;
BEGIN
    pr_resumo_pontos_tarefas;
END;
/

PROMPT === TESTE PROCEDIMENTO 2: Caso de Exceção Tratada (Sem Fatos Cadastrados) ===;
BEGIN
    SAVEPOINT sp_teste_proc2;
    -- Limpa temporariamente a tabela de fatos na sessão para induzir e_sem_fatos_cadastrados
    DELETE FROM tarefa;

```

Script Output x

Task completed in 0,08 seconds

```

=== TESTE PROCEDIMENTO 2: Execução com Subtotais Manuais e Total Geral ===
Pet (Cat 1)      Status (Cat 2)      Pontos
-----
1 - Thor         1 - PENDENTE       50.00
1 - Thor         2 - CONCLUIDO      25.00
1 - Thor         3 - EXPIRADO       15.00
Sub Total              90.00
2 - Mel          1 - PENDENTE       20.00
2 - Mel          2 - CONCLUIDO      60.00
Sub Total              80.00
3 - Bob          1 - PENDENTE       30.00
Sub Total              30.00
4 - Mimi         2 - CONCLUIDO      20.00
Sub Total              20.00
5 - Luna         1 - PENDENTE       15.00
Sub Total              15.00
Total Geral          235.00

PL/SQL procedure successfully completed.

=== TESTE PROCEDIMENTO 2: Caso de Exceção Tratada (Sem Fatos Cadastrados) ===
Pet (Cat 1)      Status (Cat 2)      Pontos
-----
Exceção capturada com sucesso: ORA-20013: Nenhum registro de tarefa/fato localizado para sumarização.

```

Trigger — trg_audit_tarefa

Os três eventos DML são disparados sobre um registro temporário (id_tarefa = 99) e os registros resultantes são consultados na tabela auditoria_dml_tarefa.

SQL

-- INSERT

```
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao,
criacao, prazo, conclusao, pet_id_pet, status_id_status,
usuario_id_usuario)
VALUES (99, 'Treino de Truque', 40, 'Ensinar comando rolar',
SYSTIMESTAMP, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 1, 1, 1);
```

-- UPDATE

```
UPDATE tarefa SET pontos_tarefa = 50, status_id_status = 2 WHERE
id_tarefa = 99;
```

-- DELETE

```
DELETE FROM tarefa WHERE id_tarefa = 99;
COMMIT;
```

-- Evidência

```
SELECT id_auditoria, nome_usuario, tipo_operacao,
data_hora_operacao, valores_antiores, valores_novos FROM
auditoria_dml_tarefa ORDER BY id_auditoria ASC;
```

Caso	Resultado Esperado
INSERT	Registro com valores_antiores = null, valores_novos preenchido
UPDATE	Registro com :OLD e :NEW preenchidos
DELETE	Registro com valores_antiores preenchido, valores_novos = null

Prints

Enzo x

0,491 seconds

Worksheet Query Builder

```
-- INSERT
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao, prazo,
                    conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (99, 'Treino de Truque', 40, 'Ensinar comando rolar',
        SYSTIMESTAMP, SYSTIMESTAMP + INTERVAL '1' DAY, NULL, 1, 1, 1);

-- UPDATE
UPDATE tarefa SET pontos_tarefa = 50, status_id_status = 2 WHERE id_tarefa = 99;

-- DELETE
DELETE FROM tarefa WHERE id_tarefa = 99;

COMMIT;

-- Evidência
SELECT id_auditoria, nome_usuario, tipo_operacao,
       data_hora_operacao, valores_antiores, valores_novos
FROM auditoria_dml_tarefa
ORDER BY id_auditoria ASC;
```

Script Output x Query Result x

Task completed in 0,491 seconds

1 row inserted.

1 row updated.

1 row deleted.

Commit complete.

>>Query Run In:Query Result

Script Output x Query Result x

All Rows Fetched: 3 in 0,032 seconds

ID_...	NOME_US...	TIPO_...	DATA_HORA_OPERACAO	VALORES_ANTERIORES	VALORES_NOVOS
1	1 RMS61432	INSERT	11-SEP-26 05.07.3...	(null)	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=40; STATUS=1; PET=1; USUARI...
2	2 RMS61432	UPDATE	11-SEP-26 05.07.3...	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=40; STATUS=1; PET=1; USUARI...	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=50; STATUS=2; PET=1; USUARI...
3	3 RMS61432	DELETE	11-SEP-26 05.07.3...	ID_TAREFA=99; TITULO=Treino de Truque; PONTOS=50; STATUS=2; PET=1; USUARI...	(null)